

Asteriskのダイヤルプラン設定編



はじめに

はじめに



今回はAsteriskのダイヤルプラン設定について説明します。

DBやAGIを使用しない極めて基本的な、ダイヤルプランの前提知識について説明します。



Asteriskとは?



Asteriskとは?

Asterisk（アスタリスク）は、アメリカアラバマ州のディジウム (Digium, Inc.) が開発していたオープンソースのIP-PBXのソフトウェア。ネーミングはアスタリスクマーク（*）に由来する

1999年に初版がリリース

※ 現在は Sangoma がメンテナンス主体

<https://www.asterisk.org>

<https://github.com/asterisk/asterisk>



ダイアルプランとは



Asteriskのダイヤルプランは、Asteriskが受け取った電話番号やコマンドをどのように処理するかを定義するルールの集合

ダイヤルプランは、Asteriskがどのように通話をルーティングし、どのようなアクションを実行するかを決定する

Asteriskの根幹となる機能であり、Asteriskを使用する上で最も重要な要素の一つ

通常 `/etc/asterisk/extensions.conf` に定義される



ダイアルプランの構成



Asteriskのダイヤルプランは、大きく以下の要素で構成される

1. **コンテキスト (Context)**: ダイヤルプランの基本的な単位で、関連するルールをグループ化します。コンテキストは、特定の電話番号やコマンドに対して適用されるルールを定義
2. **エクステンション (Extension)**: 電話番号やコマンドを表す要素で、特定のアクションを実行するためのルールを定義します。エクステンションは、コンテキスト内で定義
3. **アクション (Action)**: エクステンションに関連付けられたコマンドや機能で、通話のルーティングや処理を実行します。アクションは、エクステンション内で定義

コンテキスト: <https://docs.asterisk.org/Configuration/Dialplan/Contexts-Extensions-and-Priorities/#dialplan-contexts>

エクステンション: <https://docs.asterisk.org/Configuration/Dialplan/Contexts-Extensions-and-Priorities/#dialplan-extensions>

アクション: <https://docs.asterisk.org/Configuration/Applications/>

コンテキストの役割

[内線ユーザー用]



context: users

[外線接続用]



context: external

[IVR用]



context: ivr

同じ拡張子番号でも、コンテキストが違えば別の処理ができる

拡張子 (Extension) の概念

拡張子 = マッチさせたい番号のパターン。パターンマッチングは常に `_` から始まる

パターンマッチングで使用する特殊文字

文字	意味
<code>x</code>	任意の数字 (0～9)
<code>z</code>	任意の数字 (1～9)
<code>N</code>	任意の数字 (2～9)
<code>.</code>	任意の文字列 (1文字以上)
<code>[]</code>	文字クラス (例: <code>[13-6]</code> は 1, 3, 4, 5, 6 のいずれか)
<code>!</code>	0文字以上の文字に即座に一致 (例: <code>_9!</code> は 9 で始まる任意の文字列)

パターンマッチングの例

パターン	マッチする番号	マッチしない番号
201	201	202, 2010
_20X	201～209	200, 211
_2XX	200～299	199, 300
_2.	21, 29 , 2abc など（任意1文字以上）	2
_[1-5]XX	100～599	099, 600

Asteriskはこれらのパターンを上から順番に照合します。

パターンマッチングは「より“具体的”で、マッチ範囲が狭いパターンが勝つ」と考えてください。

例:

アリスが内線6421にダイヤルするとして、以下のダイヤルプランからどの行がマッチするでしょうか？

```
exten => _6XX1,1,SayAlpha(A)
exten => _64XX,1,SayAlpha(B)
exten => _640X,1,SayAlpha(C)
exten => _6.,1,SayAlpha(D)
exten => _64NX,1,SayAlpha(E)
exten => _6[45]NX,1,SayAlpha(F)
exten => _6[34]NX,1,SayAlpha(G)
```

優先度 (Priority) の概念

アプリケーション実行時の「順序番号」

[context]

```
exten => 200,1,Answer()           ; 優先度1: 応答  
exten => 200,2,Playback(hello)    ; 優先度2: 音声再生  
exten => 200,3,Hangup()           ; 優先度3: 切断
```

各行は必ず連続した番号を持つ必要がある

n を使った書き方

```
exten => 200,1,Answer()  
exten => 200,n,Playback(hello)    ; n = 次の番号 (2)  
exten => 200,n,Hangup()          ; n = 次の番号 (3)
```

実務では **n** を使う方が便利です。



アクション (Action) の概念

エクステンションにマッチしたとき、実際に**何をするか**を定義するもの

Asteriskでは「アプリケーション」とも呼ばれる

```
exten => 拡張子, 優先度, アクション(引数)
;                               ↑ここがアクション
exten => 200,1,Answer()
exten => 200,n,Playback(hello-world)
exten => 200,n,Hangup()
```

よく使うアクション

アクション	説明
Answer()	着信に応答する
Hangup()	通話を切断する
Dial(PJSIP/alice)	指定した相手に発信する
Playback(filename)	音声ファイルを再生する
Background(filename)	音声再生しながら入力を待つ
WaitExten(秒)	拡張子の入力を待つ
NoOp(テキスト)	何もしない（ログ・デバッグ用）
GotoIf(\$[条件]?true:false)	条件分岐

变数



変数を使用して情報を保存し、ダイヤルプラン内で利用できます。変数は、ユーザー定義のものとAsteriskが自動的に提供するものがある

<https://docs.asterisk.org/Configuration/Dialplan/Variables/>



グローバル変数

グローバル変数は、特定のチャンネルにのみ存在する変数ではなく、システム上のすべての通話に関係する変数

定義方法は以下の通り

[globals] セクションにグローバル変数を定義

```
[globals]
```

```
MY_VAR=Hello, World!
```

ダイヤルプランの GLOBAL() 関数と Set() アプリケーションを使用して、ダイヤルプランロジックからグローバル変数を設定

```
exten=>6124,1,Set(GLOBAL(MYGLOBALVAR)=somevalue)
```

組み込み変数

Asteriskには定義済の組み込み変数が多数存在する。基本的には着信を受け取ったときに自動的に設定される

また、`Dial()` や `Hangup()` などのアクションを実行したときに自動的に設定される変数もある

ここではかなり頻繁に使用する組み込み変数をいくつか紹介

標準チャンネル変数一覧: <https://docs.asterisk.org/Configuration/Dialplan/Variables/Global-Variables-Basics/>

アプリケーションの戻り値一覧: <https://docs.asterisk.org/Configuration/Dialplan/Variables/Channel-Variables/Asterisk-Standard-Channel-Variables/Application-return-values/>

変数	説明	例
<code>\${EXTEN}</code>	ダイヤルされた番号	1001
<code>\${CONTEXT}</code>	現在のコンテキスト名	users
<code>\${CHANNEL}</code>	チャネル識別子	PJSIP/alice-00000001
<code>\${UNIQUEID}</code>	通話のユニークID	1700000000.1
<code>\${PRIORITY}</code>	現在の優先度	2
<code>\${CALLERID(num)}</code>	発信者の電話番号	090xxxxxxxxx
<code>\${CALLERID(name)}</code>	発信者の名前	Alice
<code>\${DIALSTATUS}</code>	Dial() の結果	ANSWER / BUSY / NOANSWER
<code>\${PLAYBACKSTATUS}</code>	Playback() の結果	SUCCESS / FAILURE



まとめ



- Asteriskのダイヤルプランは、コンテキスト、エクステンション、アクションで構成される
- パターンマッチングを使用して、ダイヤルされた番号に対して適切な処理を定義できる
- 優先度を使用して、複数のアクションを順序立てて実行できる
- 変数を使用して、通話に関する情報を保存し、ダイヤルプラン内で利用できる

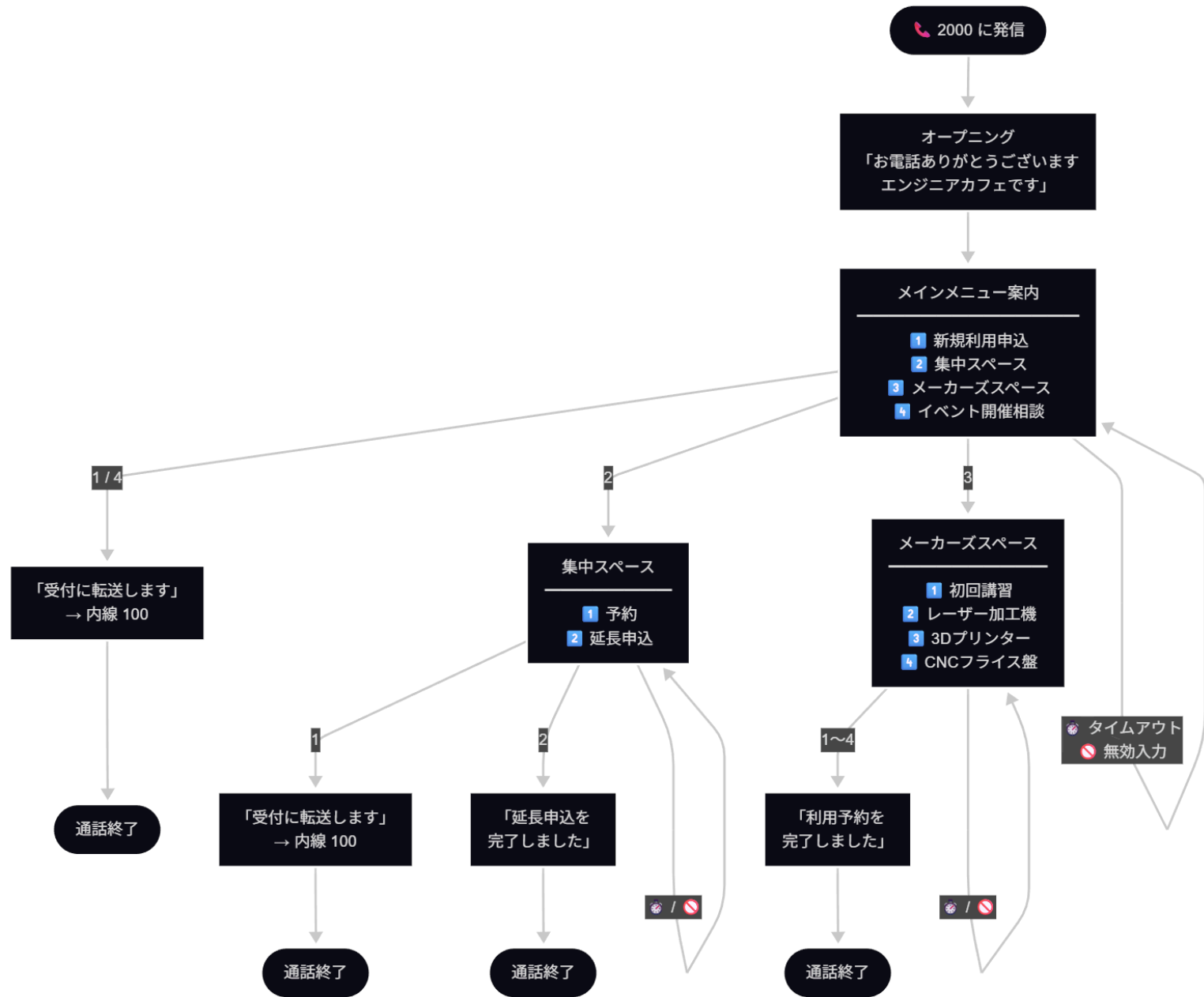
おまけ 実際に動かしてみる



ここからは、「エンジニアカフェの受付システム」を想定して、IVRを体験します。
特にガイダンス中に外部のアプリケーションに問い合わせるようなことはせず、ダイヤルプランだけで完結させる構成を例に説明します。

今回のシチュエーションは「申し込み」に焦点を当てた内容になります。





来そうな質問

Q: 音質悪っ ^^;

A: ㊄_㊄

`pjsip.conf` で使用可能なコーデックに `alaw` , `ulaw` などを指定していると音質が悪くなる。(これらのコーデックはPSTN回線などで使用されるコーデックのため、サンプリングレートが8000Hzで音質が悪い)

`pjsip.conf` の `[endpoint]` セクションで、より高品質なコーデック (例: `opus` , `g722`) を指定することで、音質を改善できる。

複数のコーデックを指定していると、`allow=` の記述順によって意図しないコーデックが選ばれることがある。`opus` だけ指定すれば确实。

Q: DTMFの方式は `rfc4733` でいいのか?

A: MicroSIPを始めとするソフトフォン、スマートフォン、IP電話機の多くは `rfc4733` 方式をサポートしているため、**一般的には `rfc4733` を使用することが推奨される。**

しかし、**お互いがサポートしていること前提になるので**、相手がアナログ電話機などを使用している想定の場合は、`inband` 方式を使用することもある。

- `rfc4733` = DTMFをRTPのイベントパケットとして送る方式。よく使われる方式で、広くサポートされている。
 - <https://tex2e.github.io/rfc-translater/html/rfc4733.html>
- `inband` 方式は物理的に音を送る方式で、相手がアナログ電話機などを使用している場合に有効。
 - 昔「電話機使って声で110番にかける」みたいなヤツがあったが、この方式を使用していると実現可能
 - <https://ja.wikipedia.org/wiki/DTMF>

Q: 内線発信として、番号以外でもできるの？

A: できます。

```
; アルファベットで始まる番号にマッチさせる例  
; この例では任意の一文字以上のアルファベットの単語のフォネティックコードを再生する  
exten => _[A-Za-z]!,1,NoOp(アルファベット番号: ${EXTEN})  
    same => n,Answer()  
    same => n,SayPhonetic(${EXTEN})  
    same => n,Hangup()
```